DURCHEX

NFT MARKETPLACE

FULL PROJECT SPECIFICATION

OpenSea-equivalent feature set • Rarible-style Explore • Real Web3 Lazy Minting

Next.js 14 App Router | MongoDB Atlas | Solidity / EVM | wagmi + RainbowKit | IPFS | Purple x Black Design System

Prepared for: Alexander Okoji
Version 1.0 — July 06, 2026

CONFIDENTIAL DRAFT

Table of Contents

PART ONE

Executive Summary & Vision

What Durchex is, who it's for, and why it can compete with OpenSea and Rarible

1. Executive Summary

Durchex is a full-stack, production-grade NFT marketplace that matches OpenSea's feature depth (browsing, collections, auctions, offers, activity feeds, profiles, rankings) while adopting Rarible's bold, editorial Explore page layout. The platform is built with Next.js 14 (App Router) end-to-end — server-rendered pages, API route handlers, and server actions — backed by MongoDB Atlas for all off-chain state (users, collections, listings, activity, social graph) and real Solidity smart contracts on an EVM chain (Polygon recommended for low fees) for ownership, transfers, and true lazy minting.

The defining technical feature is **real lazy minting**: creators sign an EIP-712 voucher off-chain (free, no gas) describing a not-yet-minted token. The voucher is stored in MongoDB and rendered on the Explore/collection pages as a normal listing with an 'Unminted' badge. When a buyer purchases it, the marketplace contract validates the signature and atomically mints the token directly to the buyer while splitting funds between the seller, the creator (royalty via EIP-2981), and the platform fee — all in one transaction, exactly like OpenSea's original Wyvern/Seaport lazy-mint pattern.

Visually, Durchex uses a Purple x Black identity with a tactile, three-dimensional design language: elevated glassmorphic cards, soft purple glow shadows, isometric 3D category icons, subtle tilt-on-hover micro-interactions, and gradient-mesh hero backgrounds — distinct from OpenSea's flatter blue/white UI and closer to Rarible's density, but with a darker, more premium finish.

1.1 Goals

- Full feature parity with OpenSea's core marketplace loop: create → list → discover → buy/bid → own → resell.
- Real on-chain settlement with off-chain-signed lazy minting, so creators can list thousands of items with zero upfront gas.
- A stylized, 3D, purple/black Explore experience inspired by Rarible's grid density and live filtering.
- A single Next.js + MongoDB codebase that is easy to run locally, deploy to Vercel, and extend.
- Multi-chain-ready architecture (starts on one EVM testnet/mainnet, contract + indexer designed to add more later).

1.2 Non-Goals (v1)

- Fiat on-ramp / credit-card checkout (flagged as a Phase 2+ integration point, e.g. MoonPay).
- Mobile native apps — v1 is a responsive web app (mobile web included in the design system).
- Cross-chain bridging of assets between networks.

- A full DAO/governance token — out of scope unless requested later.

2. Technology Stack

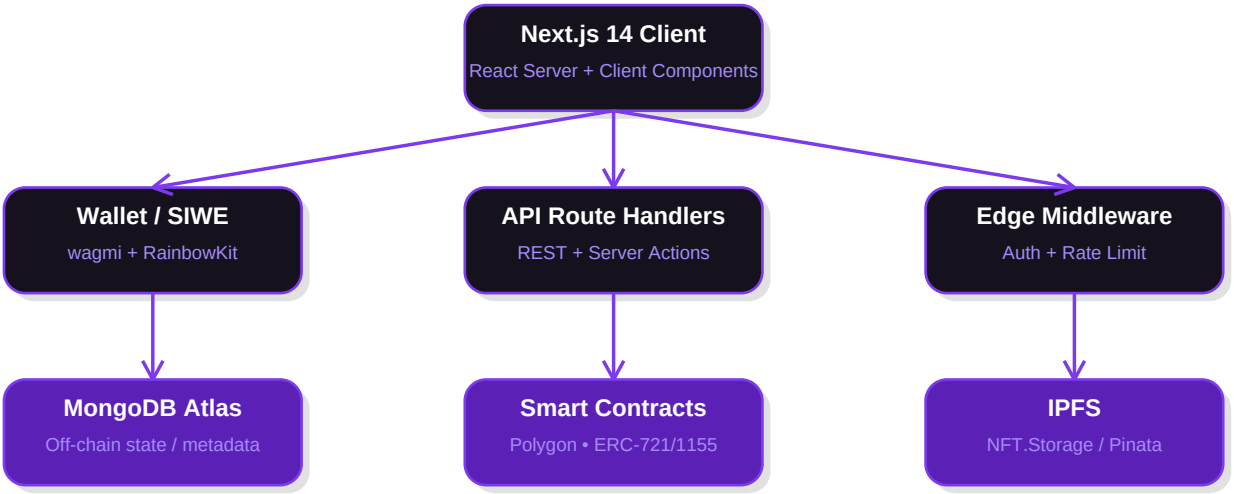
Layer	Choice	Why
Framework	Next.js 14 (App Router, RSC)	Full-stack: pages, API routes, server actions, ISR/SSR in one codebase
Language	TypeScript (strict)	Type safety across API, DB models, and contract ABIs
Database	MongoDB Atlas + Mongoose	Flexible schema for metadata/traits, fast iteration, geo/text search
Caching	Redis (Upstash)	Hot Explore queries, rate limiting, session/nonce storage
Smart Contracts	Solidity 0.8.x, OpenZeppelin, Hardhat	Audited base contracts (ERC-721/1155, EIP-2981, ReentrancyGuard)
Chain	Polygon PoS (mainnet) / Amoy (testnet)	Low gas so lazy-mint redemption is cheap for buyers
Wallet	wagmi + viem + RainbowKit	Multi-wallet connect (MetaMask, WalletConnect, Coinbase Wallet)
Auth	Sign-In With Ethereum (SIWE) + NextAuth	Wallet-native auth, no passwords, session cookie for API calls
Storage	IPFS via NFT.Storage / Pinata	Immutable metadata + media, referenced by tokenURI
Media CDN	Next/Image + Cloudflare Images	Optimized thumbnails for the Explore grid
Search	MongoDB Atlas Search (Lucene)	Fuzzy name/collection/trait search with facets
Styling	Tailwind CSS + CSS variables + Framer Motion	Design tokens for purple/black theme + 3D hover motion
Icons	Custom isometric 3D icon set (SVG) + Phosphor Icons (line set)	Distinct 3D category art, consistent line icons for UI chrome
Hosting	Vercel (app) + MongoDB Atlas + a small worker host (Railway/Fly.io) for the indexer	Serverless front end, always-on worker for chain events
Testing	Vitest/Jest, Playwright, Hardhat/Foundry tests	Unit, e2e, and contract test coverage

2.1 Monorepo Layout

```
durchex/
├── apps/
│   └── web/                    # Next.js 14 app (App Router)
│       ├── app/                # routes: (marketing)/, explore/, collection/, assets/, create/, profile/...
│       ├── app/api/            # route handlers (REST-style JSON endpoints)
│       ├── components/         # UI: cards, widgets, 3D icon set, nav, modals
│       ├── lib/                # db.ts, auth.ts, contracts.ts, ipfs.ts, indexer-client.ts
│       ├── models/             # Mongoose schemas
│       └── styles/             # tailwind.config.ts, tokens.css
├── packages/
│   ├── contracts/              # Hardhat project: DurchexNFT.sol, DurchexMarketplace.sol
│   │   ├── contracts/
│   │   ├── test/
│   │   └── scripts/deploy.ts
│   └── indexer/                # Node worker: listens to chain events, writes to MongoDB
└── docs/                       # this specification, ADRs, diagrams
```

3. System Architecture

Durchex is split into three cooperating systems: the Next.js application (UI + API), MongoDB (fast, mutable off-chain state), and the blockchain (source of truth for ownership and funds). A lightweight indexer worker keeps MongoDB in sync with on-chain truth by listening for contract events.



Chain-Event Indexer worker listens for Transfer / Sale / Redeem events and syncs MongoDB

Why a separate indexer?

API route handlers should never block a user request on a slow RPC call. Writes that must be instant (favoriting, following, off-chain voucher creation) go straight to MongoDB. Writes that depend on chain truth (a sale finalizing, a transfer happening) are only trusted once the indexer confirms the event, which prevents the UI from showing a 'sold' item that actually failed on-chain.

3.1 Request Flow — Browsing (read path)

- Client requests /explore → Next.js Server Component queries MongoDB directly (no extra API hop) for listings, using Atlas Search for filters/sort.
- Card data (image, price, collection, 'unminted' flag) is fully resolved server-side and streamed with React Suspense for fast first paint.
- Client-side widgets (live price ticker, countdown timers, favorite counts) hydrate and poll a lightweight /api/stats/live endpoint every few seconds.

3.2 Request Flow — Buying a Lazy-Minted Item (write path)

- Client calls POST /api/orders/checkout with the listing id.
- Server validates the stored EIP-712 voucher signature and current price/expiry against MongoDB, then returns calldata for the marketplace contract's redeemVoucher (fixed price) or fulfillOrder (auction/offer accept).
- Client's wallet (wagmi) sends the transaction; user pays gas + item price (or just gas if price is escrowed already via an accepted offer).

- On confirmation, the indexer worker picks up the Transfer + Sale events, marks the MongoDB listing as sold/minted, writes an Activity document, and invalidates the Explore cache tag for that collection.

PART TWO

Data & Contracts

MongoDB schemas and the Solidity contracts that back real lazy minting

4. MongoDB Data Model

All collections below are Mongoose schemas. Addresses are stored lowercase; monetary values are stored as strings (wei) to avoid floating point errors, with a derived formatted value computed at render time.

4.1 User

```
// models/User.ts
{
  _id: ObjectId,
  address: string,          // lowercase EVM address, unique, indexed
  username: string,         // unique, editable
  bio: string,
  avatarUrl: string,
  bannerUrl: string,
  isVerified: boolean,      // blue check
  socials: { twitter, discord, website, instagram },
  nonce: string,            // rotates each login, used by SIWE
  followerCount: number,
  followingCount: number,
  createdAt, updatedAt: Date
}
```

- Indexes: { address: 1 } unique, { username: 1 } unique, text index on username/bio for search.

4.2 Collection

```
// models/Collection.ts
{
  _id: ObjectId,
  slug: string,              // unique, used in /collection/[slug]
  name: string,
  description: string,
  contractAddress: string,   // deployed DurchexNFT clone/collection
  chainId: number,
  standard: "ERC721" | "ERC1155",
  creator: ObjectId,         // ref User
  category: string,          // Art, PFP, Gaming, Music, Photography, Sports, ...
  logoUrl, bannerUrl, featuredUrl: string,
  royaltyBps: number,        // basis points, e.g. 500 = 5%
  verified: boolean,
  stats: {
    floorPrice: string, volume24h: string, volume7d: string, totalVolume: string,
    owners: number, items: number, sales: number
  },
  createdAt, updatedAt: Date
}
```

- Indexes: { slug: 1 } unique, { 'stats.volume7d': -1 } for rankings, Atlas Search index on name/category.

4.3 Item (NFT)

```
// models/Item.ts
{
  _id: ObjectId,
  collection: ObjectId,       // ref Collection
  tokenId: string | null,     // null until minted (lazy item)
  isMinted: boolean,
  owner: ObjectId,            // ref User; = creator until first sale if lazy
  creator: ObjectId,
  name: string,
  description: string,
  imageUrl: string,           // IPFS-backed, resolved via gateway
  animationUrl: string | null,
  metadataUri: string,        // ipfs://... (source of truth once minted)
  traits: [{ trait_type: string, value: string, rarity: number }],
  status: "not_listed" | "fixed_price" | "auction" | "sold",
  favoriteCount: number,
  viewCount: number,
  voucher: {                  // present only while isMinted = false
    tokenId: string, uri: string, minPrice: string,
    creator: string, royaltyBps: number, signature: string, nonce: number
  },
  createdAt, updatedAt: Date
}
```

- Indexes: { collection: 1, tokenId: 1 } unique, { status: 1 }, { 'traits.trait_type': 1, 'traits.value': 1 } for filter facets.

4.4 Listing

```
// models/Listing.ts
{
  _id: ObjectId,
  item: ObjectId,
  seller: ObjectId,
  type: "fixed_price" | "auction",
  price: string,               // wei, fixed_price only
  currency: "MATIC" | "WETH",
  startTime: Date, endTime: Date | null,
  reservePrice: string,        // auction only
  highestBid: ObjectId,        // ref Bid, auction only
  status: "active" | "cancelled" | "filled" | "expired",
  createdAt, updatedAt: Date
}
```

4.5 Bid / Offer

```
// models/Bid.ts   (also used for collection & item Offers)
{
  _id: ObjectId,
  listing: ObjectId | null,     // set for auction bids
  item: ObjectId | null,        // set for direct item offers
  collection: ObjectId | null,  // set for collection-wide offers
  bidder: ObjectId,
  amount: string,
  currency: "WETH",
  expiresAt: Date,
  status: "active" | "accepted" | "rejected" | "expired" | "cancelled",
  signature: string,             // off-chain signed order (Seaport-style) until accepted
  createdAt, updatedAt: Date
}
```

4.6 Activity

```
// models/Activity.ts
{
  _id: ObjectId,
  type: "mint" | "list" | "sale" | "transfer" | "bid" | "offer" | "cancel",
  item: ObjectId,
  from: ObjectId, to: ObjectId,
  price: string | null,
  txHash: string | null,        // null for pure off-chain events (e.g. new listing)
  createdAt: Date
}
```

- Powers the global Activity page and per-item/per-collection history tabs; capped/aged out or archived after 1 year.

4.7 Supporting collections

```
// Favorite { user, item, createdAt }              — unique on (user,item)
// Follow   { follower, following, createdAt }      — unique on (follower,following)
// Notification { user, type, payload, read, createdAt }
// Report   { reporter, targetType, targetId, reason, status }
// Category { key, label, iconKey }                — drives the 3D icon Explore tabs
```

5. Smart Contracts

Two contracts power the chain side. **DurchexNFT** is an ERC-721 collection contract with a lazy-mint redeem function and EIP-2981 royalties. **DurchexMarketplace** handles fixed-price sales, English auctions, and signed offers, and is the only address allowed to call redeemVoucher (so payment and mint always happen atomically).

5.1 DurchexNFT.sol (lazy-mint voucher redemption)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "@openzeppelin/contracts/token/ERC721/extensions/ERC721URIStorage.sol";
import "@openzeppelin/contracts/token/common/ERC2981.sol";
import "@openzeppelin/contracts/utils/cryptography/EIP712.sol";
import "@openzeppelin/contracts/utils/cryptography/ECDSA.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

contract DurchexNFT is ERC721URIStorage, ERC2981, EIP712, Ownable {
    using ECDSA for bytes32;

    struct NFTVoucher {
        uint256 tokenId;
        string  uri;
        uint256 minPrice;      // wei, floor the marketplace must honor
        address creator;
        uint96  royaltyBps;
        uint256 nonce;         // replay protection per creator
    }

    address public marketplace;                // only this address may redeem
    mapping(uint256 => bool) public minted;
    mapping(address => uint256) public nonces;

    bytes32 private constant VOUCHER_TYPEHASH = keccak256(
        "NFTVoucher(uint256 tokenId,string uri,uint256 minPrice,address creator,uint96 royaltyBps,uint256 nonc
    );

    constructor() ERC721("Durchex", "DRX") EIP712("Durchex", "1") Ownable(msg.sender) {}
```

```
        function setMarketplace(address _m) external onlyOwner { marketplace = _m; }

        function _hash(NFTVoucher calldata v) internal view returns (bytes32) {
            return _hashTypedDataV4(keccak256(abi.encode(
                VOUCHER_TYPEHASH, v.tokenId, keccak256(bytes(v.uri)),
                v.minPrice, v.creator, v.royaltyBps, v.nonce
            )));
        }

        function redeem(address buyer, NFTVoucher calldata voucher, bytes calldata signature)
            external returns (uint256)
        {
            require(msg.sender == marketplace, "only marketplace");
            require(!minted[voucher.tokenId], "already minted");
            require(voucher.nonce == nonces[voucher.creator], "bad nonce");

            address signer = _hash(voucher).recover(signature);
            require(signer == voucher.creator, "invalid signature");

            minted[voucher.tokenId] = true;
            nonces[voucher.creator]++;
            _mint(voucher.creator, voucher.tokenId);          // mint to creator...
            _setTokenURI(voucher.tokenId, voucher.uri);
            _setTokenRoyalty(voucher.tokenId, voucher.creator, voucher.royaltyBps);
            _transfer(voucher.creator, buyer, voucher.tokenId); // ...then straight to buyer
            return voucher.tokenId;
        }

        function supportsInterface(bytes4 interfaceId)


            public view override(ERC721URIStorage, ERC2981) returns (bool)
        { return super.supportsInterface(interfaceId); }
    }
```

5.2 DurchexMarketplace.sol (settlement, fees, auctions)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "@openzeppelin/contracts/security/ReentrancyGuard.sol";
import "@openzeppelin/contracts/token/ERC721/IERC721.sol";
import "@openzeppelin/contracts/token/common/ERC2981.sol";
import "./DurchexNFT.sol";

contract DurchexMarketplace is ReentrancyGuard {
    uint96 public constant PLATFORM_FEE_BPS = 250;      // 2.5%, matches OpenSea-era norms
    address public feeRecipient;

    event VoucherRedeemed(address indexed nft, uint256 indexed tokenId, address buyer, uint256 price);
    event ListingFilled(address indexed nft, uint256 indexed tokenId, address seller, address buyer, uint256 p

    constructor(address _feeRecipient) { feeRecipient = _feeRecipient; }

    // --- Lazy-mint fixed-price purchase ---
    function buyLazy(
        DurchexNFT nft,
        DurchexNFT.NFTVoucher calldata voucher,
        bytes calldata signature
    ) external payable nonReentrant {
        require(msg.value >= voucher.minPrice, "insufficient payment");
        uint256 tokenId = nft.redeem(msg.sender, voucher, signature);
        _settle(voucher.creator, voucher.creator, voucher.royaltyBps, msg.value);
        emit VoucherRedeemed(address(nft), tokenId, msg.sender, msg.value);
    }

    // --- Already-minted fixed price sale (standard resale) ---


    function buyListed(IERC721 nft, uint256 tokenId, address seller, uint256 price) external payable nonReentr
        require(msg.value >= price, "insufficient payment");
        require(nft.ownerOf(tokenId) == seller, "seller no longer owns token");
        nft.safeTransferFrom(seller, msg.sender, tokenId);
        (address royaltyReceiver, uint256 royaltyAmt) = ERC2981(address(nft)).royaltyInfo(tokenId, price);
        _settle(seller, royaltyReceiver, uint96(royaltyAmt * 10000 / price), price);
        emit ListingFilled(address(nft), tokenId, seller, msg.sender, price);
    }

    // --- English auction settlement (called by highest bidder or seller after endTime) ---
    function settleAuction(IERC721 nft, uint256 tokenId, address seller, address winner, uint256 amount) exter
        nft.safeTransferFrom(seller, winner, tokenId);
        (address royaltyReceiver, uint256 royaltyAmt) = ERC2981(address(nft)).royaltyInfo(tokenId, amount);
        _settle(seller, royaltyReceiver, uint96(royaltyAmt * 10000 / amount), amount);
    }

    function _settle(address seller, address royaltyReceiver, uint96 royaltyBps, uint256 amount) internal {
        uint256 fee = (amount * PLATFORM_FEE_BPS) / 10000;
        uint256 royalty = (amount * royaltyBps) / 10000;
        uint256 sellerProceeds = amount - fee - royalty;
        payable(feeRecipient).transfer(fee);
        if (royalty > 0 && royaltyReceiver != seller) payable(royaltyReceiver).transfer(royalty);
        payable(seller).transfer(sellerProceeds + (royaltyReceiver == seller ? royalty : 0));
    }
}
```

Security notes

> Both contracts use OpenZeppelin's audited primitives, ReentrancyGuard on all payable settlement paths, checks-effects-interactions ordering, and per-creator nonces to prevent voucher replay. Full details in Section 15 (Security).

6. Lazy Minting — End-to-End Flow

This is the flow that makes Durchex behave exactly like OpenSea's original lazy-mint experience: creators list unlimited items with zero gas, and the very first buyer's transaction performs the mint.

6.1 Creating a Lazy Listing

- Creator uploads media + fills metadata in /create → media pinned to IPFS, tokenURI JSON pinned, imageUrl cached via CDN.
- Client builds an NFTVoucher (tokenId reserved sequentially per collection, uri, minPrice, royaltyBps, nonce) and asks the wallet to sign it via EIP-712 (eth_signTypedData_v4) — this is free, no transaction.
- POST /api/items creates the Item document with isMinted:false and the voucher + signature embedded, plus a 'not_listed' or 'fixed_price' Listing.
- Item immediately appears on Explore/collection pages with an 'Unminted' badge — identical query path to minted items, just a boolean flag difference.

6.2 Buying a Lazy Item

- Buyer clicks Buy Now → client fetches the stored voucher + signature from MongoDB via GET /api/items/[id]/voucher.
- Client calls DurchexMarketplace.buyLazy(nft, voucher, signature) with msg.value = price via wagmi's useWriteContract.
- Contract verifies the signature, mints the token straight to the buyer, and splits funds (seller / royalty / platform fee) atomically — impossible to pay without receiving the NFT, and vice versa.
- Indexer sees VoucherRedeemed, flips isMinted → true, sets owner, writes an Activity 'sale' record, and revalidates the collection's ISR cache tag so stats (floor, volume, owners) update within seconds.

> **What if two buyers click Buy Now at once?**
>
> The contract's `minted[tokenId]` guard makes the second transaction revert on-chain — the mempool/gas race decides the winner, and MongoDB is only updated after indexer confirmation, so the UI never shows a false positive.

PART THREE

Application Design

API surface, auth, sitemap, and every page in the product

7. API Route Handlers

Method & Path	Auth	Purpose
GET /api/explore	—	Paginated, filterable, sortable feed (status, price range, chains, collections, traits, category)
GET /api/collections	—	List/search collections with stats, supports sort by volume/floor
GET /api/collections/[slug]	—	Collection detail + paginated items + trait facets
POST /api/collections	SIWE	Create a collection (deploys or registers a clone contract)
GET /api/items/[id]	—	Item detail: metadata, traits, owner, listing, offers, activity
POST /api/items	SIWE	Create item (mints now, or stores lazy voucher if deferred)
GET /api/items/[id]/voucher	SIWE	Returns signed voucher payload for on-chain redeem
POST /api/listings	SIWE	Create fixed-price or auction listing
DELETE /api/listings/[id]	SIWE (owner)	Cancel a listing
POST /api/bids	SIWE	Place an auction bid or a direct offer (off-chain signed order)
POST /api/bids/[id]/accept	SIWE (seller)	Seller accepts an offer → returns settlement calldata
POST /api/favorites/[itemId]	SIWE	Toggle favorite
POST /api/follow/[address]	SIWE	Toggle follow
GET /api/users/[address]	—	Public profile: created, owned, collected, activity
PATCH /api/users/me	SIWE	Update profile (username, bio, avatar, socials)
GET /api/activity	—	Global or scoped (item/collection/user) activity feed
GET /api/rankings	—	Top collections by 1h/24h/7d/30d volume
GET /api/stats/live	—	Lightweight polling endpoint for tickers/counters
POST /api/auth/nonce	—	Issues SIWE nonce for a wallet address
POST /api/auth/verify	—	Verifies signed SIWE message, sets session cookie
POST /api/webhooks/indexer	Service token	Internal: indexer worker pushes confirmed chain events

8. Authentication — Sign-In With Ethereum

- Connect Wallet (RainbowKit) → client requests a nonce from POST /api/auth/nonce for the connected address.
- Client builds a SIWE message (domain, address, nonce, chainId, expiration) and requests eth_signTypedData / personal_sign from the wallet.
- POST /api/auth/verify validates the signature server-side, upserts the User document, and sets an httpOnly session cookie (NextAuth Credentials provider wrapping SIWE).
- All mutating API routes read the session server-side; no client-supplied address is ever trusted for authorization.

9. Site Map

Route	Description
/	Home — full marketing + discovery landing page (see Section 11)
/explore	Rarible-style filterable grid of all items (see Section 12)
/drops	Curated upcoming/live launches with countdowns
/rankings	Top collections leaderboard, sortable by timeframe and chain
/collection/[slug]	Collection banner, stats bar, trait filter sidebar, item grid, activity tab
/assets/[chainId]/[contract]/[tokenId]	Item detail: media viewer, price panel, Buy/Bid, offers table, properties, activity, more-from-collection
/create	Single-item mint/list wizard (upload → traits → price → sign voucher)
/create/collection	Collection creation wizard (name, symbol, royalties, category, socials)
/profile/[address]	Owned / Created / Favorited / Activity tabs, editable if viewing own profile
/activity	Global live activity ticker with filters (sales, listings, offers, mints)
/search	Full-text results across items, collections, and users
/notifications	Offers received, bids outbid, sales, follows
/settings	Profile edit, connected socials, notification preferences
/login	Wallet connect + SIWE entry point (also available as a modal anywhere)

PART FOUR

Design System

Purple x Black identity, 3D elevation model, typography, and components

10. Visual Identity — Purple x Black

Token	Hex	Usage
--bg-void	#0B0A10	App background, nav, footer — near-black, not pure black
--bg-surface	#15121E	Card/panel base surface
--bg-surface-2	#1E1930	Raised panel / modal / dropdown
--purple-900	#3B0A85	Deep gradient stop, pressed states
--purple-700 (Primary)	#7C3AED	Primary buttons, links, active nav, focus rings
--purple-500	#A78BFA	Secondary text on dark, icon strokes
--purple-300	#C4B5FD	Hover highlight, badge backgrounds
--accent-pink-purple	#C084FC	Gradient accent stop for hero/glow effects
--text-onlight	#1A1523	Body text on white cards (light sections)
--text-muted	#8A82A0	Secondary/metadata text
--success	#22C55E	Price up, minted confirmation
--danger	#F43F5E	Price down, delist, errors

10.1 Typography

- **Display / Headings:** Clash Display (Fontshare, free) — geometric, bold, distinctive uppercase treatment for hero numerals and section titles.
- **Body / UI:** Inter — highly legible at small sizes for card metadata, tables, and forms.
- **Numerals / Prices:** Inter with tabular-nums variant so price columns align in tables and the activity feed.
- Scale: Display 56/64, H1 40/48, H2 28/36, H3 20/28, Body 15/24, Small 13/20, Micro 11/16 (all in px/line-height).

10.2 The 3D Elevation Model

Every interactive surface uses a consistent shadow + gradient recipe to feel tactile without becoming heavy:

- **Level 0 (flat):** no shadow, used for the page background and section dividers.

- **Level 1 (card):** 1px inner top highlight (rgba(255,255,255,0.06)) + soft drop shadow 0 8px 24px rgba(124,58,237,0.18) on a #15121E surface — the base NFT/collection card.
- **Level 2 (hover/raised):** card translateY(-6px) scale(1.02) with shadow growing to 0 20px 40px rgba(124,58,237,0.35) and a subtle 6deg 3D tilt (perspective + rotateX/rotateY driven by cursor position via Framer Motion) — this is the signature 'stunning' hover moment.
- **Level 3 (modal/popover):** stronger blur backdrop (glassmorphism, backdrop-filter: blur(20px)) with a 1px purple-tinted border and heavier shadow.
- **Glow accents:** primary CTAs and live-auction badges use a pulsing radial purple glow (box-shadow animation) rather than flat color, reinforcing the premium/energetic feel.

10.3 3D Icon Set

Category and utility icons (Art, PFP, Gaming, Music, Photography, Sports, Virtual Worlds, Wallet, Lightning/Gas, Verified Badge, Auction Gavel) are custom isometric SVG icons rendered with two-tone purple gradients and a soft drop shadow to look like small floating 3D objects — distinct from the flat line-icon set (Phosphor) used for generic UI chrome (search, menu, chevrons, close).

10.4 Core Components

Component	Notes
NFTCard	Media (auto-play short video/GIF on hover), collection name + verified tick, item name, price + USD estimate, favorite heart w/ count, 'Unminted' or 'Live Auction' ribbon, quick-buy button on hover
CollectionCard	Banner + circular logo overlap, name, floor price, 24h volume delta (green/red), item count
StatWidget	Big numeral (Clash Display) + label + trend arrow, used in hero stats band and rankings
CountdownTimer	Monospace tabular digits, purple glow when < 1 hour remains
TraitPill	Rounded pill, trait_type in muted text over value in bold, shows rarity % on hover
WalletChip	Avatar + truncated address/ENS + balance, dropdown for profile/settings/disconnect
FilterSidebar	Collapsible sections: Status, Price, Chains, Collections (checkbox list w/ counts), Traits (facet counts)
Navbar	Glass-blur sticky bar, global search with instant dropdown results, Explore/Stats/Create links, Connect Wallet button
Toast/TxTracker	Bottom-right stacked toast showing pending → confirmed states for on-chain transactions with a block-explorer link

11. Home Page — Section-by-Section

The home page is intentionally long and rich, mixing marketing storytelling with live marketplace data so it never feels like a static template.

#	Section	Content
1	Announcement bar	Slim dismissible bar for a featured drop or platform news, purple-on-black gradient
2	Sticky Navbar	Logo, Explore/Rankings/Drops/Stats links, global search, Create button, Connect Wallet
3	Hero	Large Clash Display headline, gradient-mesh purple/black background, floating 3D-tilted NFT card stack that parallax-rotates with mouse movement, primary CTAs (Explore / Create), and a live stats strip (Total Volume, Items, Creators, Floor avg)
4	Trending Collections	Horizontally scrollable carousel of CollectionCards ranked by 24h volume, with a live-updating volume delta
5	Live Auctions	Grid of NFTCards currently in auction with CountdownTimer and current highest bid
6	Top Creators / Sellers	Leaderboard widget (avatar, name, volume) with 1D/7D/30D toggle
7	Featured Drops	Large editorial cards for curated upcoming launches with a 'Notify me' action
8	Browse by Category	Grid of the 3D isometric category icons (Art, PFP, Gaming, Music, Photography, Sports, Virtual Worlds) linking into pre-filtered Explore views
9	How Lazy Minting Works	3-step visual explainer (Sign → List → Buyer Mints) reusing the voucher-flow diagram style, aimed at creators
10	Platform Stats Band	Full-width dark band with big numerals: Total Volume, NFTs Minted, Active Wallets, Collections
11	Testimonials / Press	Quote cards from creators + logo strip
12	Newsletter / Creator CTA	Split panel: email capture on one side, 'Start Creating' CTA with gradient card on the other
13	Footer	Sitemap columns, social links, chain/network badge, copyright

12. Explore Page — Rarible-Style Layout

The Explore page favors density and live data over the marketing feel of the home page, mirroring Rarible's approach: a persistent filter rail, a category tab strip, and a tight, responsive card grid.

- **Category tab strip** at the top (All, Art, PFPs, Gaming, Music, Photography, Sports, Virtual Worlds) using the 3D icon set at small size, horizontally scrollable on mobile.
- **Left filter sidebar** (collapsible on mobile into a bottom-sheet): Status (Buy Now, On Auction, New, Has Offers), Price range with min/max + currency toggle, Chains, Collections (searchable checkbox list with per-collection item counts), Traits (accordion per trait_type with value counts, auto-generated per collection).
- **Toolbar**: result count, Sort dropdown (Price: Low→High/High→Low, Recently Listed, Recently Created, Most Favorited, Ending Soon), grid density toggle (compact/comfortable), view toggle (grid/list).
- **Card grid**: responsive 2/3/4/6-column grid depending on viewport, each NFTCard using the Level 1→2 hover elevation, quick-buy button revealed on hover, live price with USD conversion.
- **Infinite scroll** with a skeleton-loading state matching the card shape (shimmer in purple tones, not gray) and a 'Back to top' floating button once scrolled.
- **Empty/no-results state** uses a friendly 3D illustration (floating disconnected purple cube) rather than plain text.

> **Why server components here too**
>
> Filters are encoded as URL search params, so the initial grid for any filter combination is server-rendered (shareable/bookmarkable links, good SEO for collection/category pages) and only subsequent infinite-scroll pages are fetched client-side from /api/explore.

13. NFT Detail Page

- Large media viewer (image/video/3D model via for glb files) with zoom.
- Title, collection link + verified badge, owner + creator chips, chain badge.
- Price panel: current price (or highest bid + countdown), Buy Now / Place Bid / Make Offer buttons, 'Unminted — mints on purchase' note when applicable.
- Tabs: Properties (TraitPills with rarity %), Offers (table of active offers with Accept for the owner), Activity (mint/list/sale/transfer history), Details (contract address, token ID, token standard, chain, metadata link).
- 'More from this collection' carousel at the bottom.

14. Create / Mint Flow

- Step 1 — Upload media (drag/drop, accepts image/video/audio/3D, shows IPFS pin progress).
- Step 2 — Details: name, description, external link, collection picker (or create new inline), properties/traits editor with rarity auto-calculation preview.
- Step 3 — Supply & pricing: unlockable content toggle, explicit/sensitive content flag, Instant Sale price or Auction (start price/reserve/duration), or save as unlisted draft.
- Step 4 — Review & Sign: summary card styled exactly like the live NFTCard preview, single wallet signature (EIP-712, free) to create the lazy voucher — no gas paid here.
- Confirmation screen with a shareable link and a 'List another' shortcut.

PART FIVE

Non-Functional Requirements

Security, performance, testing, and delivery plan

15. Security Considerations

- **Signature/replay protection:** per-creator nonces on vouchers; off-chain offers use EIP-712 typed data with expiry timestamps checked both server-side and on-chain at settlement.
- **Reentrancy:** all payable settlement functions use OpenZeppelin ReentrancyGuard and checks-effects-interactions.
- **Access control:** only the registered marketplace address may call DurchexNFT.redeem; collection creation validates the caller owns/deploys the contract before indexing it as 'verified-pending'.
- **Session security:** httpOnly, sameSite=strict session cookies from SIWE verification; nonce single-use and short-lived (5 minutes).
- **Rate limiting:** Redis-backed limits on write endpoints (listing creation, bids, follows) to deter spam/wash trading.
- **Content moderation:** automated NSFW/malware scan on upload before IPFS pin, manual report queue (Report model) feeding an admin review dashboard.
- **Front-running/mempool:** auction settlement uses a commit-reveal-free simple highest-bid model acceptable for v1; documented as an upgrade path to a sealed-bid design if needed.
- **Escrow:** auction bids beyond the first are represented off-chain (signed, not funded) until settlement, avoiding locked funds — mirrors OpenSea/Seaport's offer model rather than fully-funded escrow auctions.

16. Performance & Scalability

- Home and marketing sections use ISR (revalidate: 60s); Explore/collection pages are dynamic with `fetch` cache tags invalidated by the indexer on relevant events.
- MongoDB compound indexes on (collection, status), (traits.trait_type, traits.value), and Atlas Search index for text + facet queries used by Explore/search.
- Redis cache for rankings and stats widgets (10–30s TTL) to absorb polling load from the live ticker components.
- Images served via next/image with IPFS-gateway → Cloudflare Images pipeline; blurred low-res placeholders generated at upload time.
- Indexer worker batches RPC calls and uses websocket subscriptions (viem's watchContractEvent) instead of polling for near-real-time updates.

17. Testing Strategy

Layer	Tooling	Coverage
Smart contracts	Hardhat + Foundry	Voucher redemption, replay protection, fee/royalty math, reentrancy attempts, access control
API routes	Vitest + mongodb-memory-server	Auth guards, validation, pagination, filter correctness
Components	React Testing Library	Card states (minted/unminted/auction), filter sidebar interactions
End-to-end	Playwright	Connect wallet (mock signer) → create lazy listing → buy → appears in profile

18. Deployment & Infrastructure

- Frontend/API: Vercel (Next.js native), preview deployments per PR.
- Database: MongoDB Atlas (M10+), automated backups, Atlas Search index managed via IaC (mongodb-atlas Terraform or Atlas CLI scripts checked into the repo).
- Contracts: Hardhat deploy scripts, verified on Polygonscan, addresses recorded in a versioned deployments.json consumed by the app.
- Indexer: small always-on Node worker (Railway/Fly.io) with a health-check endpoint monitored by uptime alerting.
- Secrets: Vercel/host environment variables for RPC URLs, WalletConnect project id, IPFS pinning keys, session secret — never committed.

19. Roadmap

Phase	Scope
0 — Foundation	Repo scaffold, design tokens, MongoDB models, wallet connect + SIWE auth
1 — MVP	Create/mint (lazy + real), collection pages, item detail, fixed-price buy, profile, Explore grid + filters
2 — Marketplace depth	Auctions, offers/bidding, activity feed, rankings, notifications, search
3 — Polish	3D icon set, motion/hover system, home page storytelling sections, performance passes
4 — Growth	Featured drops/curation tools, creator verification flow, admin moderation dashboard
5 — Expansion	Additional chains, fiat on-ramp integration, mobile-optimized PWA

20. Feature Parity Checklist

Capability	OpenSea	Rarible	Durchex v1
Lazy minting (gasless listing)	Yes (legacy)	Partial	Yes — real EIP-712 voucher redeem
Fixed-price buy	Yes	Yes	Yes
English auctions	Yes	Yes	Yes
Offers / collection offers	Yes	Yes	Yes
Rarity traits & facets	Yes	Yes	Yes
Activity feed	Yes	Yes	Yes
Rankings / stats	Yes	Yes	Yes
Verified collections	Yes	Yes	Yes
Notifications	Yes	Partial	Yes
Stylized 3D / dense Explore grid	No (flatter UI)	Yes (inspiration)	Yes — purple/black 3D take

End of specification. Implementation begins with Phase 0 (repo scaffold, design tokens, MongoDB models, wallet connect) as agreed.